

Lab04 Fun with Threads

This lab gives you a chance to explore programming using threads. Threads can be great and make your program run faster, but threads can also cause new problems and the programs can be harder to debug because it is difficult to replicate errors when using threaded code.

For the reason of un-replicability, the automated checkpoint system cannot be used for this lab. For this lab you must document the checkpoints in a file and submit the final file (a PDF) to FLO. You should capture the output of the program and discuss how the checkpoint was completed (code snippets encouraged). There are suggestions for what to include in your submission. You will also need to submit your final code to FLO.

Task A: Unsynchronously Expanding Array

You have been provided with skeleton code (see `cp3.lab04.expandablearray`) that provides a class implementing an expandable array, `ExpandableArray`, similar to that from figure 6.24 of the book *Parallel Programming: For Multicore and Cluster Systems*. It works in a similar way to a `Vector` or an `ArrayList`, but has a poorly implemented expansion function (it only increments the size of the array by 2 when expansion is required) and bad things will happen if the expanding array is accessed or modified by more than one thread. Here is the `add` method:

```
public void add(Object x)
{
    if (size == data.length) // too small
    {
        Object[] od = data;
        //data = new Object[3 * (size + 1)/2];
        data = new Object[size + 2]; // poor expansion value
        System.arraycopy(od, 0, data, 0, od.length);
    }
    data[size++] = x;
}
```

Checkpoint L4.1

Your first task is to write some code to explore how the expanding array fails when accessed and modified by multiple threads. The class  `ExpandableArrayDriver` creates an `ExpandableArray` called `ea` that has an initial size of 1 and is filled up with 10 Integers. This should cause the `ea` to expand a number of times.

You successfully achieve the checkpoint when your program has lots of runtime errors!

HINTS

You should create another class that extends the `Thread` class (or implements `Runnable`, you choose) and

takes and modifies an `ExpandableArray` object. You might want to pass the `ExpandableArray` object `ea` to the constructor of your new class. This will allow multiple instantiations of your class to share the same instance of an `ExpandableArray` object – thus allowing for bad things to happen! Bad things will happen when you perform expanding array operations such as fill the array beyond the capacity, remove from the array, getting from the array and doing it concurrently with multiple threads.

Information to include for Checkpoint Submission

Copy the output from several different executions of the program demonstrating how the program can

- run without crashing
- crash!
- crash in a different way

You should discuss how you created a class to cause the program to crash due to shared memory between threads.

Checkpoint L4.2

Your task now is to explore how to use the Java synchronization techniques for making our expanding array be more thread-safe (but still run in parallel).

You successfully achieve the checkpoint when your program has no more errors!

HINTS

For example, the expandable array can be successfully filled up by multiple threads with a somewhat expected result. The result is only *somewhat* expected as you do not want to turn your parallel, supadupa threaded execution into a boring old serial execution by blocking each thread until the prior one completes, but you must still allow multi-threaded access to the array.

For example, if you have 3 threads inserting 1000 integers then it should be possible to see in the sequence

..., 99, 54, 100, 55, 1, 2, 101, ...

where, for example, thread 1 added 99, 100, 101, thread 2 added 54 55, and thread 3 added 1, 2. The calls to add have been successfully interleaved due to your most awesome synchronisation techniques.

If, however, you see in the output an unbroken sequence of 1 to 1000, then another 1 to 1000 and then another 1 to 1000, then each thread has sequentially added to the array!

This may be due to the fact that the threads start and finish before the other threads have even started! In that case, make the number of integers added larger or add more threads to cause congestion. If that does not help, then it might be because you have serialised your implementation by not so good use of synchronisation techniques! For example, excessive use of the thread method `.join()`.

Information to include for Checkpoint Submission

Copy the output from several different executions of the program demonstrating how the program can now run successfully and still allows for parallel access to the `ExpandableArray`. For example:

- non-sequential integers in the output
- an execution with a different sequence

You should discuss your thread-policy (the use of Java thread synchronisation techniques) and how this still allows for parallel access to the `ExpandableArray`.

Task B: File Encryption/Decryption

Let's assume that you and I have important secrets, and we want to keep them from prying eyes. We shall do so by creating a tool for encrypting our most secret files. Encryption is the process of making data unreadable to anyone except those possessing a key; for our purposes, this key will be a password with which the file is encrypted – and with which we can ultimately decrypt that file.

Naturally, we have rather many secrets, stored in many large files, and because people with many secrets are very busy, we want our tool to be as fast as possible. The objective of this practical is to develop a file encryption tool that uses multithreading to maximize its encryption and decryption performance. The tool is to be developed in Java, and you will hence gain some experience with Java threads.

Implementing encryption in Java is not difficult, as the Java API provides extensive support for this task. However, to save you some time, I have provided a substantial program skeleton that you can just fill in.

The project contains the following sub-packages:

1. `cp3.lab04.crypt`: which contains 2 classes: 
`JCrypt` and `JCryptUtil`
2. `jargs.gnu`: which is used by `JCryptUtil` for command-line option parsing (and you can pretty much ignore it)

You are asked to edit  `JCrypt` to complete the checkpoints of this lab. It makes extensive use of the static methods and nested classes in `JCryptUtil`, which you might want to have a look at (but do not need to modify).

Checkpoint L4.3

You can then run  JCrypt using the  JCrypt no args run configuration or from the command-line as follows:

```
java cp3.lab04.crypt.JCrypt
```

This will give you usage instructions for  JCrypt, which can already encrypt and decrypt files. It's just not very fast at doing so. Using the JCrypt encrypt txt run configuration or the following command line

```
java cp3.lab04.crypt.JCrypt -e MAGIC -s temp.txt
```

will encrypt the text file temp.txt in the current directory using the password MAGIC, assuming you had that file in the directory. The -s option here ensures output to a file; otherwise it ends up on standard out. The encrypted files are stored with the suffix .encrypted.

Download the file prac4-secrets.zip from FLO and extract into this project directory (so that you have a directory called prac4-secrets).

Please do your best to NOT submit these files back to FLO.

We shall decrypt some of my secret files now by using the JCrypt decrypt secrets1 run configuration which expects the encrypted files to be in a directory called prac4-secrets of the IntelliJ working directory, or from the command line:

```
java cp3.lab04.crypt.JCrypt -d enigma -s prac4-secrets/*.jpg.encrypted
```

You will need to wait a few seconds for this to complete. You should now have a bunch of highly confidential JPEG images in your directory. JCrypt will inform you how long it took to decode the files.

Your first job is to use Java threads to make it go faster.

Extend  JCrypt so that it encrypts or decrypts each file on a separate thread. You only need to modify the main(...) method for this, but you are welcome to apply larger changes or restructure the code to your liking. Additionally, remember to

use the `Thread.join()`, locks, barrier synchronisation or some other technique to ensure that all the threads have finished before you print out the timing information, as we want to accurately observe how much faster  JCrypt can run with threads.

HINTS

- Create a new subclass of `Thread` that overrides the `run()` method, so that it calls `process(...)`. Keep in mind that `process(...)` expects some parameters, so you need to somehow pass them into your new thread object (e.g., via a constructor).
- Instead of calling `process(...)` in the for-loop of the `main(...)` method, you have to instantiate your new thread subclass and call its `start()` method.
- To ensure that all the threads are finished before the print statement at the end of the program, you should add every thread you instantiate to a list. You then iterate through that list and call `join()` on every thread. You might also want to explore using barriers or maybe even a countdown latch!

Information to include for Checkpoint Submission

You should detail your thread-policy (how you achieve thread greatness) and note how much faster your threaded implementation is over the original serial implementation.

Checkpoint L4.4

Having a thread for each file leads to a lot of threads for a lot of files, but you are wasting resources if the number of threads greatly exceeds the number of processor cores.  JCrypt therefore accepts a `-t` option, followed by a number, as a command-line argument:

```
java flinders.crypt.JCrypt -d enigma -t 4 -s  
    prac4-secrets/*.jpg.encrypted
```

or by using the `JCrypt decrypt secret t4` run configuration.

The `-t` option is expected to allow you to specify the **maximum number of threads** that  `JCrypt` will use at any given time. If you have more files than threads, each of the existing thread should be reused once it is done encoding or decoding a file. Implementing this into  `JCrypt` is your task for checkpoint 21.

HINTS

- The number of threads specified on the command-line is stored in `opts.threads`.
- You need to create the specified number of threads and change the `run()` method so that when it finishes processing one filename it moves on to the next (that is, the next filename that isn't already being processed by another thread right now).
- The `process(...)` method takes, as a second parameter, the number of the file in the list of command-line filenames to process. Thus, the simplest approach is to store the lowest number that hasn't been assigned to a thread so far. You can create a counter for this, which you pass to every thread, so that they know which filenames haven't been processed so far (i.e., all the ones above the counter's value). *Make sure access to the counter is thread-safe!*
- When `process(...)` finishes in a thread, you (thread-asfe) increment the counter and use its value as the next argument to `process(...)`.
- You should consider putting synchronization around any modification or comparison operation involving the counter, as it is shared between threads (I'm making this too easy).
- Alternatively, you could explore the use of thread pools (yes do that, pools are fun).

Information to include for Checkpoint Submission

You should detail your thread-policy (how you achieve thread greatness) and note how much faster your constrained threaded implementation is over the original serial and threaded implementations.

If you have more 8 or more cores, then try making duplicates of the encrypted files to give the cores some work to do. If you do you will need to update the run configuration with the new file names.